

软件工程课程作业

软件设计报告

校缘聚 (CampusGather) 桌游/剧本杀组局平台

团队序号：第 1 组

项目名称	校缘聚 (CampusGather) 桌游/剧本杀组局平台
设计依据	《校缘聚需求分析文档》与本次软件设计报告要求
交付内容	软件设计报告、UML/ER/架构图、可运行原型、数据库脚本与测试

姓名	学号	组内说明
刘砚桐	2313983	学生端与组局业务设计
李佳璞	2314007	架构、后端与报告整合
苏雨辰	2313828	需求校对与用户流程设计
史傅冠华	2311987	管理端、投诉与信用设计
朱乐晨	2312194	场地管理与数据库设计

2026 年 6 月 6 日

目 录

1 系统总体设计	2
1.1 项目背景与目标	2
1.2 系统功能概述	2
1.3 系统总体架构设计	2
1.4 技术选型说明	3
2 详细设计	4
2.1 模块划分与模块功能设计	4
2.2 模块调用关系	5
2.3 核心业务流程设计	5
2.4 接口设计	7
3 UML 设计	9
3.1 用例图	9
3.2 类图	10
3.3 顺序图	10
3.4 部署图	11
4 数据库设计	12
4.1 数据库 E-R 图	12
4.2 主要数据表设计	12
4.3 关键索引与一致性设计	14
5 安全、性能与可扩展性设计	15
5.1 安全性设计	15
5.2 性能优化设计	15
5.3 可扩展性设计	15
5.4 异常处理机制	16
6 实现对应关系与测试计划	17
6.1 报告设计与代码实现对应关系	17
6.2 原型边界	17
6.3 测试计划	18

1 系统总体设计

1.1 项目背景与目标

校缘聚 (CampusGather) 面向校园桌游、剧本杀等线下组局场景。需求分析阶段已经明确, 当前学生主要依赖微信群、QQ 群、贴吧等松散渠道发起活动, 常见问题包括信息分散、参与者身份难以确认、活动变更通知不及时、临时跳车难追溯以及校内活动空间申请流程不透明等。桌游与剧本杀又具有固定人数、固定时长、线下协作强和信用约束强的特点, 因此需要一个能够把“发布、招募、审核、场地、通知、评价、投诉”串联起来的系统。

本系统的设计目标如下:

1. 为普通学生提供组局浏览、筛选、报名、退出、互评、投诉与个人信用管理能力;
2. 为发起人提供组局发布、成员审核、活动状态维护和场地预约申请能力;
3. 为系统管理员提供实名认证、游戏库、违规内容、投诉、信用和日志统计管理能力;
4. 为场地/社团管理员提供场地维护、开放时段配置、预约审核、占用追踪和场地通知能力;
5. 在课程周期内完成可运行原型, 同时预留 MySQL、学校统一身份认证、微信小程序和通知推送等后续扩展点。

1.2 系统功能概述

用户域	核心功能	设计说明
普通学生	浏览/筛选组局、查看详情、申请加入、退出、互评、投诉、个人资料 (含头像与标签)、信用记录、实名认证提交	未认证用户只能浏览公开信息; 认证后才能参与影响活动与信用的操作; 可通过个人主页编辑头像、昵称、标签等。
发起人	发布组局、编辑组局、审核报名、取消组局、发起场地申请	发起人是普通学生在特定组局中的业务角色, 权限限定在自己创建的组局。
系统管理员	实名认证审核、账号状态管理、用户列表查看、游戏库、违规内容、投诉与信用、通知、日志统计	后台接口统一走角色鉴权, 所有关键操作写入管理员操作日志; 可查看全部用户列表并对用户进行认证审核与账号状态变更。
场地管理员	场地信息、开放时段、预约审核、占用记录、场地通知	只能处理被授权管理的场地, 审核结果同步影响组局场地状态。
外部系统	学校统一身份认证、通知推送、日志监控	当前原型采用模拟适配器; 部署阶段替换为真实接口。

表 1.1: 系统功能概览

1.3 系统总体架构设计

系统采用客户端-服务器与分层 MVC 相结合的架构。客户端层包含微信小程序学生端和管理后台; 应用服务层使用 Node.js REST API 承载鉴权、组局、场地、信用、投诉、通知等模块; 数据层以 Repository 抽象隔离存储实现, 开发阶段使用 JSON 文件持久化以便演示, 部署阶段可切换为 MySQL, 并通过事务、索引和外键保证一致性。

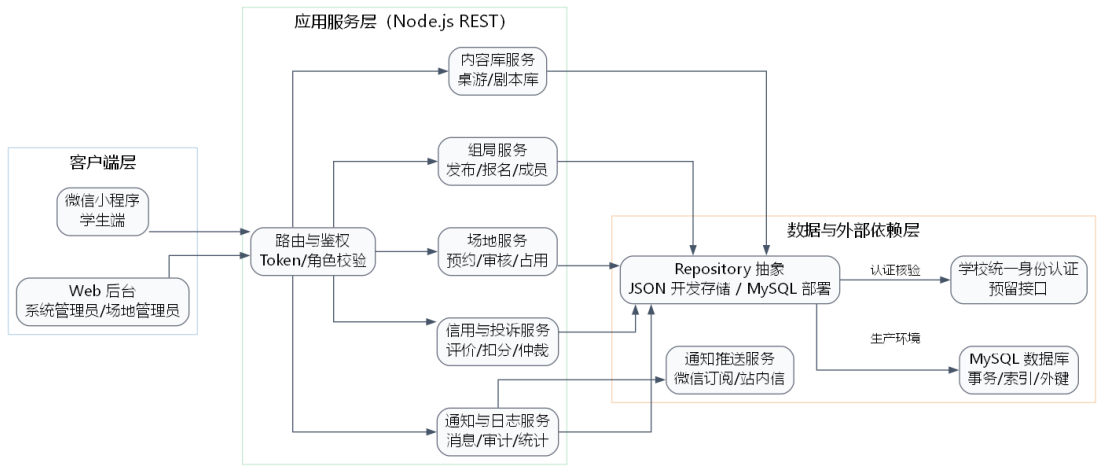


图 1.1: 系统总体架构设计

层次	职责	可实践约束
表现层	移动端与后台页面，负责表单录入、状态展示、操作确认、错误提示	前端不直接访问数据文件；所有业务操作经 REST API 完成。
接口/控制层	路由分发、参数校验、登录态解析、角色权限校验、统一响应格式	错误码统一为 VALIDATION_ERROR、FORBIDDEN、NOT_FOUND、CONFLICT 等。
业务服务层	组局、报名、场地、投诉、信用、通知、日志等业务规则	关键状态变更集中在服务层，避免前端绕过规则。
数据访问层	通过 Repository 封装 CRUD、查询、保存与种子数据	开发期 JSONRepository；生产期 MySQLRepository，接口保持一致。
基础设施层	学校认证、推送、监控、备份	均使用适配器模式，原型可模拟，部署可替换。

表 1.2: 架构分层说明

1.4 技术选型说明

类别	选型	理由
前端	微信小程序/响应式 Web 原型	符合校园移动使用场景；Web 原型便于课程验收直接运行。
后端	Node.js 原生 HTTP + 模块化服务	异步 I/O 适合高并发浏览与报名；无外部依赖，便于离线演示和部署。
接口	RESTful API + JSON	易于小程序调用、调试和生成接口文档；统一错误码便于前后端协作。

类别	选型	理由
数据库	MySQL 8.0（报告设计） JSON 文件（原型开发）	MySQL 支持事务和关系约束；JSON 存储降低课程演示环境成本，二者由 Repository 隔离。
图表	Graphviz DOT	本地可生成 UML、ER 与架构图，源文件可追踪，后续可转 PlantUML 或 draw.io。
测试	Node.js 内置 node:test	不依赖网络安装包即可覆盖核心接口与业务规则。
部署	Nginx + Node.js + MySQL	结构简单，适合校内试点；后续可通过多实例、缓存和读写分离扩展。

表 1.3: 技术选型说明

2 详细设计

2.1 模块划分与模块功能设计

模块	主要职责	输入/输出	关键规则
Auth/User	登录、当前用户、个人资料（含头像编辑）、实名认证提交与管理、用户列表管理、角色与账号状态变更	输入学号或用户编号； 输出用户视图（含头像 URL、认证提交信息、权限）	未认证用户不可发布、报名、评价、投诉；学号和联系方式仅本人和管理员可见；头像限定预设选项。
GameLib	维护桌游/剧本库，供发布和筛选使用	游戏名称、类型、人数、时长、难度、标签	下架条目不可被新组局选择。
Session	组局发布、列表筛选、详情、编辑、取消、完结	活动时间、地点、人数、信用要求、加入方式	时间必须合法；名额不得超上限；关键变更发通知。
Application/Member	报名申请、发起人审核、退出、成员名单、签到状态	申请备注、审核动作、退出原因	审核制先进入待审核；直接加入走容量和冲突校验。
Venue	场地资源、开放时段、预约申请、审核、占用记录	场地、时间、申请说明、审核意见	容量、开放状态、时间冲突和管理权限必须校验。
Review/Credit	活动互评、信用分流水、跳车或投诉扣分	评分、评价内容、信用变更原因	仅实际成员可评价；信用变更必须有业务关联。
Complaint	投诉提交、受理、驳回/成立、处理结果	投诉原因、证据、目标用户	管理员处理后同步通知双方并写日志。
Notification/Log/Stats	站内通知、已读状态、管理员操作日志、平台统计	业务事件和操作上下文	日志不可由普通接口删除；统计仅管理员可见。

模块	主要职责	输入/输出	关键规则
----	------	-------	------

表 2.4: 模块功能设计

2.2 模块调用关系

控制器只负责请求解析和响应封装，不直接改写业务数据。每个控制器调用对应服务；服务之间通过显式方法协作。例如报名成功后，SessionService 调用 NotificationService 生成通知，并调用 LogService 写入操作记录；投诉成立后，ComplaintService 调用 CreditService 写入信用流水，再通知投诉双方。

入口操作	主服务	协作服务	数据实体
发布组局	SessionService	GameLibService、VenueService、NotificationService、LogService	game_sessions、 session_members、 venue_reservations、 notifications、 admin_logs
申请加入	SessionService	UserService、CreditService、NotificationService	session_applications、 session_members、 credit_records
场地审核	VenueService	SessionService、NotificationService、LogService	venue_reservations、 venues、game_sessions
处理投诉	ComplaintService	CreditService、NotificationService、LogService	complaints、 credit_records、 notifications、 admin_logs
实名认证审核	UserService	ExternalAuthAdapter、NotificationService、LogService	users、notifications、 admin_logs
实名认证提交	UserService	—	users

表 2.5: 模块调用关系

2.3 核心业务流程设计

发布组局流程：用户认证通过后进入发布页面，选择游戏库条目并填写标题、人数、时间、场地、加入方式和最低信用分。系统校验时间合法性、人数范围、游戏状态和场地容量与时间冲突；场地选择为必填项，发布组局时自动创建已通过的场地预约记录（venue_status 置为 approved），组局地点字段自动由场地信息生成。保存组局后将发起人写入成员表。后续发起人仍可通过编辑组局更换场地或调整时间，系统自动同步场地预约。

实名认证流程：学生进入个人主页，填写真实姓名、学号和联系方式提交认证申请，auth_status 变为 pending。管理员在后台用户列表中查看待审核用户及其提交信息，选择通过（approve）或驳回

(reject) 并填写审核意见。通过后用户 auth_status 变为 verified, 获得发布、报名、评价等操作权限; 驳回后用户可修改信息重新提交。审核结果通过通知模块反馈给申请人。

报名与成员审核流程: 学生提交申请后, 系统检查认证状态、账号状态、信用分、组局状态、名额、重复报名和时间冲突。直接加入模式立即生成成员; 审核制生成待审核申请, 由发起人通过或拒绝。所有结果通过通知模块反馈。

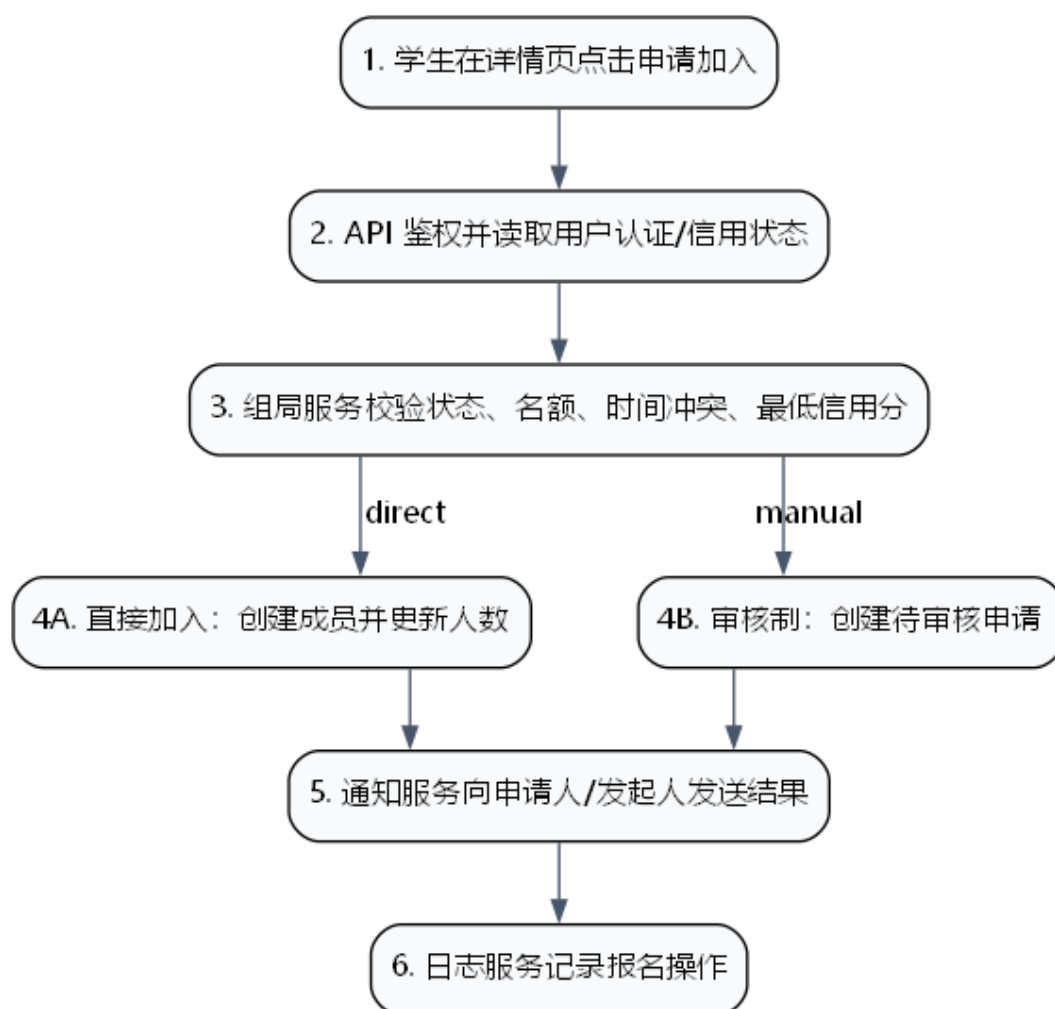


图 2.2: 申请加入组局顺序图

场地审核流程: 发起人提交预约后, 场地服务校验场地状态、容量和时间范围, 并检查已通过预约是否冲突。场地管理员只能审核自己管理的场地。通过后预约状态变为 approved, 组局详情显示场地已确认; 驳回时保存原因并通知发起人。

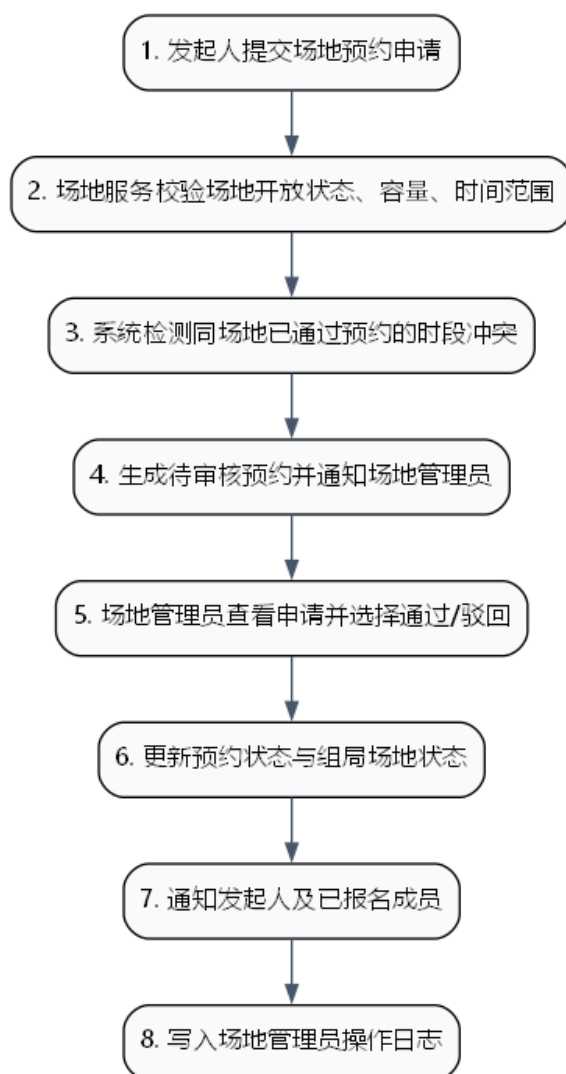


图 2.3: 场地预约审核顺序图

2.4 接口设计

方法	路径	角色	说明
GET	/api/health	公开	健康检查与版本信息
POST	/api/auth/login	公开	使用学号/角色演示登录，返回用户视图
GET	/api/users/me	登录用户	查看个人资料与权限
PUT	/api/users/me	登录用户	编辑个人资料（昵称、头像、标签、联系方式）

方法	路径	角色	说明
POST	/api/users/me/auth	学生	提交实名认证申请（真实姓名、学号、联系方式）
GET	/api/users/me/credit	登录用户	查看个人信用分与信用流水
GET	/api/users	管理员	查看全部用户列表
PATCH	/api/users/:id/auth	管理员	审核用户实名认证（通过/驳回）
PATCH	/api/users/:id/status	管理员	变更用户账号状态 (active/muted/limited/banned)
GET/POST	/api/games	公开/管理员	查询游戏库；管理员新增游戏
PATCH	/api/games/:id	管理员	更新游戏条目信息
GET/POST	/api/sessions	公开/认证学生	筛选组局；发布组局（必选场地）
GET	/api/sessions/mine	登录用户	查看我发布或参与的组局
GET/PATCH	/api/sessions/:id	公开/发起人	查看详情（含场地与成员信息）；发起人编辑组局
POST	/api/sessions/:id/applications	认证学生	申请加入组局或直接加入
PATCH	/api/applications/:id	发起人	通过/拒绝报名申请
POST	/api/sessions/:id/leave	成员	退出组局并按规则记录信用影响
POST	/api/sessions/:id/cancel	发起人	取消组局
POST	/api/sessions/:id/finish	发起人	结束组局
POST	/api/sessions/:id/reviews	成员	活动结束后互评
GET/POST/ PATCH	/api/complaints	学生/管理员	提交、查询、处理投诉
GET/POST	/api/venues	场地管理员	查询场地列表；新增场地
GET/PATCH/ DELETE	/api/venues/:id	场地管理员	查看、编辑、删除场地（删除时级联取消关联组局）
GET/POST/ PATCH	/api/venue-reservations	发起人/场地 管理员	提交、查询、审核场地预约
GET/PATCH	/api/notifications	登录用户	查看通知与标记已读
GET	/api/admin/logs	管理员	查看操作日志
GET	/api/admin/stats	管理员	查看平台统计

表 2.6: 主要 REST 接口设计

统一响应格式为:成功时返回 { success: true, data };失败时返回 { success: false, error:

{ code, message, details } }。客户端根据 HTTP 状态码和 code 显示明确错误，例如名额已满、权限不足、信用分不足、场地冲突、重复报名等。

3 UML 设计

3.1 用例图

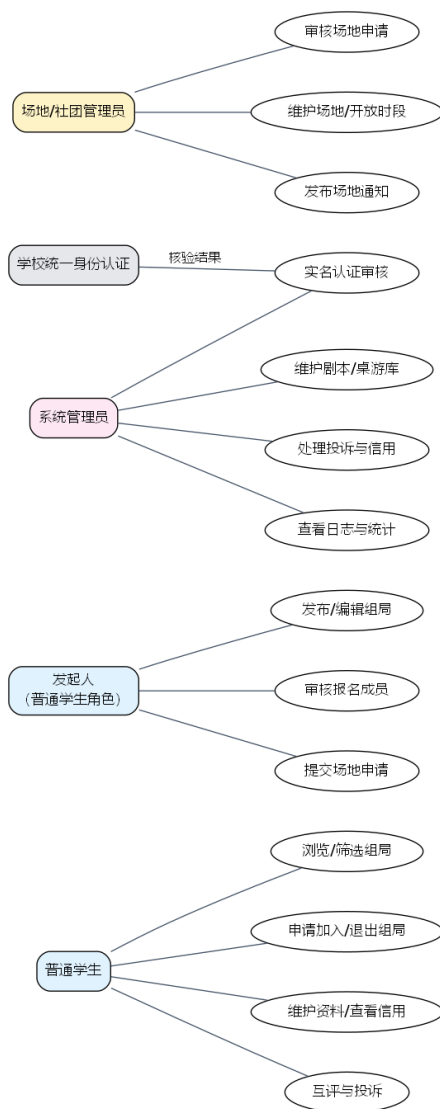


图 3.4: 系统用例图

用例图覆盖普通学生、发起人、系统管理员、场地/社团管理员与学校统一身份认证接口。发起人不是独立账号类型，而是普通学生在自己发布的组局中的临时角色。

3.2 类图

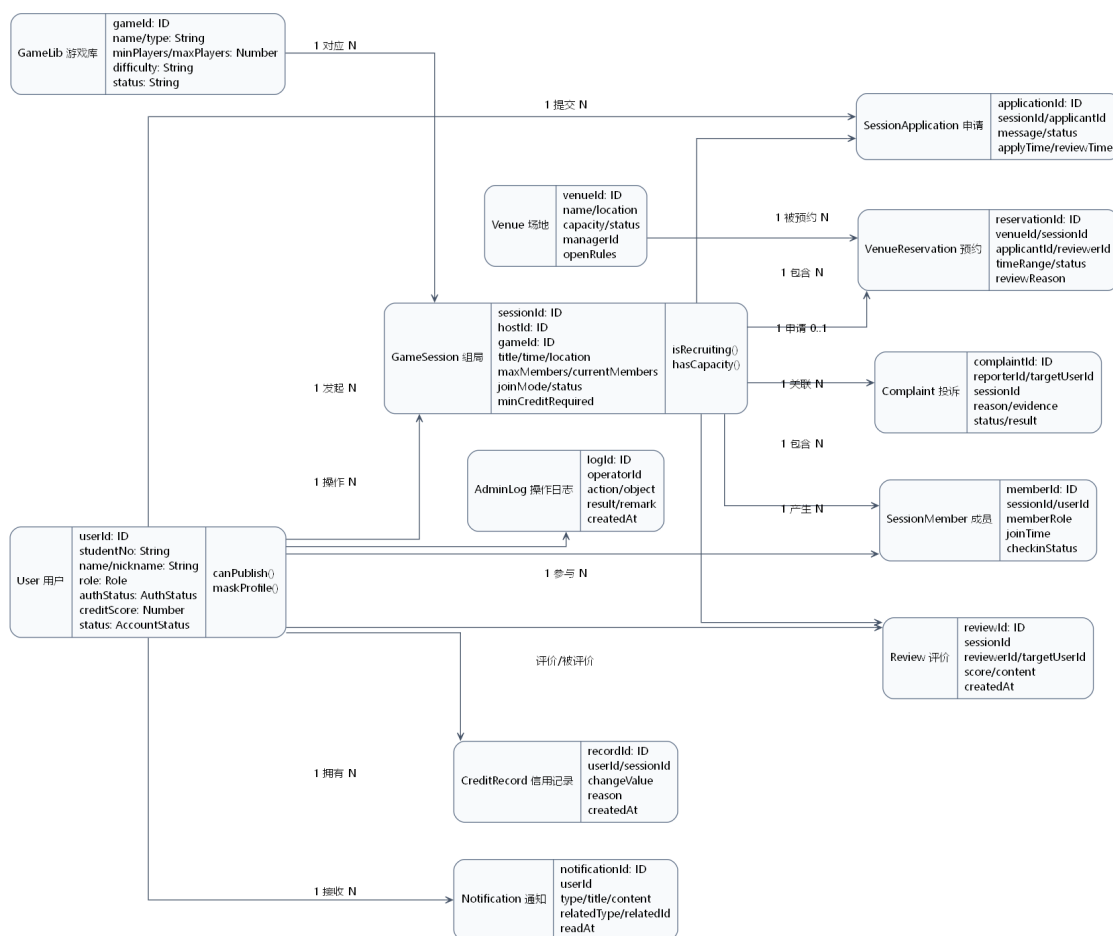


图 3.5: 核心领域类图

类图以业务领域对象为中心。User 与 GameSession、SessionApplication、SessionMember 共同支撑组局；Venue 与 VenueReservation 支撑场地审批；Review、Complaint、CreditRecord 共同支撑信用闭环；Notification 与 AdminLog 为横切能力。

3.3 顺序图

核心顺序图已在第 2 章给出，分别展示申请加入组局与场地预约审核。实现时应保证顺序图中的校验步骤在服务层执行，而不是只在前端提示。

3.4 部署图

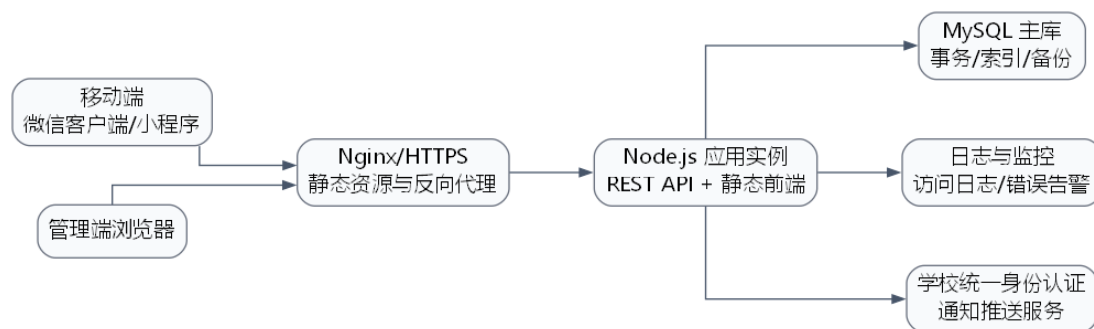


图 3.6: 系统部署图

部署设计支持课程原型和后续试点两种形态。课程原型可由一个 Node.js 进程同时提供静态页面和 API；正式试点时通过 Nginx 接入 HTTPS，Node.js 多实例连接 MySQL，并接入日志告警、学校统一身份认证和推送服务。

4 数据库设计

4.1 数据库 E-R 图

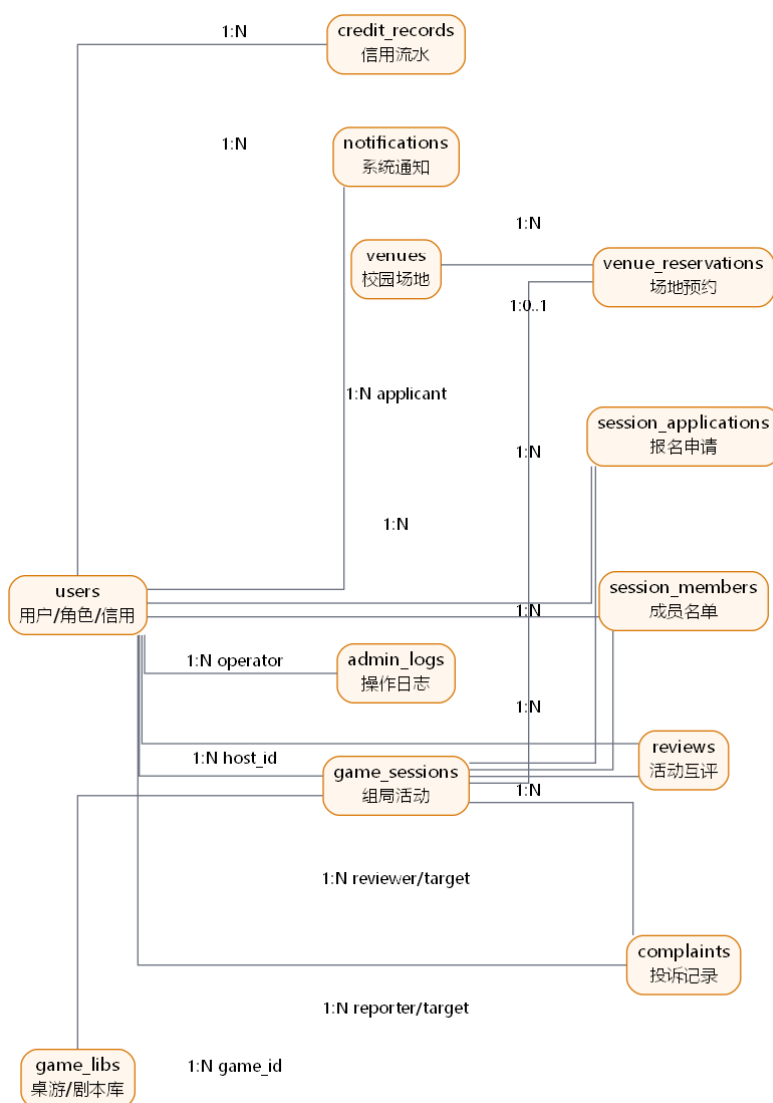


图 4.7: 数据库 E-R 图

数据库以 users 为主体，game_sessions 为活动核心。报名申请、成员、预约、评价、投诉、信用、通知、日志均通过外键关联到用户或组局，便于追溯完整业务链路。

4.2 主要数据表设计

表名	关键字段	主键/外键	说明
users	id, student_no, name, nickname, role, auth_status, credit_score, status, avatar, tags, contact, auth_submission_name, auth_submission_student_no, auth_submission_contact, auth_submission_note, auth_submitted_at, auth_review_reason, auth_reviewed_at, status_reason	PK id; UNIQUE student_no	用户、管理员与场地管理员统一建模, 通过 role 区分权限。ID 采用 5 位数字编码: 10001-11000 为管理员, 11001-99999 为普通用户。avatar 存储头像文件名, auth_submission 系列字段支持实名认证提交流程。
game_libs	id, name, type, min_players, max_players, duration_minutes, difficulty, description, tags, status	PK id	桌游/剧本库, 供发布组局和筛选使用。
game_sessions	id, host_id, game_id, title, description, start_time, end_time, location, max_members, current_members, min_credit_required, join_mode, status, venue_status, cancel_reason	PK id; FK host_id; FK game_id	一次具体组局活动。venue_status 记录场地预约状态, cancel_reason 记录取消原因。
session_applications	id, session_id, applicant_id, message, status, apply_time, review_time, review_reason	PK id; FK session_id; FK applicant_id	审核制报名申请与直接加入记录。
session_members	id, session_id, user_id, member_role, join_time, checkin_status	PK id; FK session_id; FK user_id; UNIQUE(session_id,user_id)	已加入成员名单。
venues	id, name, location, capacity, manager_id, available_time, open_rules, status, description	PK id; FK manager_id	校园场地基础数据与开放规则。
venue_reservations	id, venue_id, session_id, applicant_id, reviewer_id, start_time, end_time, status, review_reason	PK id; FK venue_id; FK session_id; FK applicant_id	场地预约审核记录。

表名	关键字段	主键/外键	说明
reviews	id, session_id, reviewer_id, target_user_id, score, content, created_at	PK id; FK session_id; FK reviewer/target	活动结束后互评。
complaints	id, reporter_id, target_user_id, session_id, reason, evidence, status, result, handled_by	PK id; FK reporter/target; FK session_id	投诉受理、处理和结果记录。
credit_records	id, user_id, session_id, complaint_id, change_value, reason, created_at	PK id; FK user_id; FK session_id; FK complaint_id	信用分变化流水。
notifications	id, user_id, type, title, content, related_type, related_id, read_at, created_at	PK id; FK user_id	站内通知与后续微信订阅消息记录。
admin_logs	id, operator_id, action, object_type, object_id, result, remark, created_at	PK id; FK operator_id	后台关键操作审计日志。

表 4.7: 主要数据表设计

4.3 关键索引与一致性设计

1. `game_sessions` 建立 `(status, start_time)`、`(game_id, start_time)` 和 `host_id` 索引, 支撑列表筛选和个人组局查询。
2. `session_members` 对 `(session_id, user_id)` 建唯一约束, 防止重复报名; `session_applications` 对待审核状态建立组合索引。
3. `venue_reservations` 对 `(venue_id, start_time, end_time, status)` 建索引, 审核时快速发现时间冲突。
4. 报名审批、退出扣分、投诉成立后的信用变更应放在同一事务中执行; 原型由单进程同步写文件保证演示一致性, 生产由 MySQL 事务保证。

5 安全、性能与可扩展性设计

5.1 安全性设计

安全点	设计方案
身份认证	所有写操作必须登录；发布、报名、评价、投诉要求 <code>auth_status=verified</code> 。学校统一身份认证采用适配器接入，原型可模拟审核。
权限控制	普通学生、系统管理员、场地管理员按角色授权；发起人只能管理自己发布的组局；场地管理员只能审核自己管理的场地。
隐私保护	学号、联系方式、投诉证据等敏感字段默认不在公开接口返回；对外展示信用记录采用次数和类型脱敏。
输入校验	标题、简介、评价、投诉内容进行长度、必填和敏感词校验；人数、时间、评分等使用类型和范围校验。
审计日志	实名认证、账号状态、违规处理、投诉处理、场地审核、信用调整等后台动作写入 AdminLog，至少保留 6 个月。

表 5.8: 安全性设计

5.2 性能优化设计

1. 列表接口支持分页和条件筛选，首页只返回必要摘要，详情页再加载完整信息。
2. 报名和审核接口采用原子校验：先查状态、名额、重复成员和时间冲突，再写入成员/申请。
3. 热门查询字段建立索引，游戏库和场地基础数据可在客户端短期缓存。
4. 通知发送采用先落库、后推送策略；推送失败不影响主业务完成，可重试。
5. 目标指标沿用需求文档：列表与筛选 2 秒内返回，关键操作 3 秒内反馈，日常支持 100 名用户同时浏览。

5.3 可扩展性设计

1. 活动类型以 `game_libs.type` 和 `tags` 扩展，后续可加入狼人杀、密室、社团活动等。
2. 角色权限集中在鉴权中间件与用户 `role` 字段，后续可加入 DM、社团负责人、商家等角色。
3. 推荐算法独立为 `RecommendationService`，初期基于时间/类型/地点筛选，后续可引入兴趣标签和历史参与记录。
4. 外部学校认证、微信订阅消息、短信或邮件都通过 `Adapter` 封装，避免业务服务直接依赖第三方 SDK。
5. `Repository` 抽象保证原型 JSON 存储和生产 MySQL 存储可以替换，接口和服务层不变。

5.4 异常处理机制

异常类型	处理策略	用户提示
参数不完整/格式错误	控制层返回 400 与字段级 details	指出具体字段，例如“活动结束时间必须晚于开始时间”。
权限不足	鉴权层返回 403 并记录异常访问	提示“当前账号无权执行该操作”。
资源不存在	服务层返回 404	提示目标组局、场地或申请不存在。
业务冲突	服务层返回 409	提示名额已满、重复报名、时间冲突或场地冲突。
外部接口失败	适配器超时重试并降级为待人工审核	提示认证/推送暂不可用，稍后重试或等待管理员处理。

表 5.9: 异常处理机制

6 实现对应关系与测试计划

6.1 报告设计与代码实现对应关系

报告模块	代码实现	验收方式
接口/控制层	src/server.js, src/router.js	访问 /api/health, 使用前端或测试脚本调用核心接口。含头像静态资源路由 /profile_photo/。
业务服务层	src/services/userService.js, sessionService.js, venueService.js, complaintService.js 等	通过 node --test 覆盖登录、认证提交与审核、发布（含场地绑定）、报名、审核、场地冲突、投诉信用变更。
数据访问层	src/database/jsonStore.js, src/database/seed.js 与 data/*.json	重启后数据不丢失；seed.js 预置 6 个用户（含头像、认证状态）、2 个场地、3 款游戏和 2 条组局示例。
数据库设计	database/schema.sql	检查表、索引、外键与本报告字段一致；users 表含 avatar 与 auth_submission 系列字段。
前端演示	public/index.html, public/app.js, public/styles.css	浏览器打开本地服务即可完成学生个人主页编辑、实名认证提交、管理员审核、场地选择发布组局、参与者详情查看等完整流程。
静态资源	data/profile_photo/（12 张头像图片）	头像通过 /profile_photo/ 文件名访问，支持 default、adm、1-10 共 12 个选项。

表 6.10: 设计与实现映射

6.2 原型边界

本次课程实现以完整业务闭环为目标，不接入真实微信 AppID、学校统一身份认证和线上推送密钥；这些环境参数需要学校或课程平台确认后替换。原型使用 JSON 文件持久化，便于课堂演示；报告已给出 MySQL 表结构，后续可将 Repository 实现替换为 MySQL。

6.3 测试计划

阶段	内容	完成标准
单元/接口测试	健康检查、登录、筛选、发布（含场地绑定）、申请、审核、退出、取消/完结、投诉、实名认证提交与审核、场地审核与删除	<code>npm test</code> 全部通过。
业务验收	学生端完成“编辑个人主页-实名认证提交-浏览-报名-评价/投诉”；管理员端完成“认证审核/用户列表/投诉/内容库”；场地端完成“场地管理/预约审核”。	前端页面可演示，头像上传与展示、ID分配方案、数据持久化可追溯。
性能检查	构造列表筛选和连续报名请求	关键接口在本地 3 秒内响应，无超额报名。
安全检查	未登录、未认证、越权账号分别调用写接口	返回 401/403，不修改数据。
提交	报告源码、图片、PDF、源码、README、schema.sql、测试	推送到 GitHub main 分支。

表 6.11: 测试与交付计划